

```
!pip install google-generativeai PyMuPDF python-dotenv pandas tqdm
```

Show hidden output

```
import google.generativeai as genai
import fitz # PyMuPDF for PDF processing
import os
import json
import csv
import re
import pandas as pd
from dotenv import load_dotenv
from google.colab import files
from tqdm import tqdm # Progress bar for batch processing

os.environ["G00GLE_API_KEY"] = "API_key"
genai.configure(api_key=os.environ["G00GLE_API_KEY"])

# Load API Key from Environment Variables
load_dotenv()
api_key = os.getenv("G00GLE_API_KEY")
if not api_key:
    raise ValueError("G00GLE_API_KEY not found in environment variables.")
genai.configure(api_key=api_key)

# Load API Key from Environment Variables
load_dotenv()
api_key = os.getenv("G00GLE_API_KEY")
if not api_key:
    raise ValueError("G00GLE_API_KEY not found in environment variables.")
genai.configure(api_key=api_key)

# Global Variables to Store Resume and Job Description
resume_text = None
job_description = None

# Function to extract text from a PDF
def extract_text_from_pdf(pdf_path):
    try:
        doc = fitz.open(pdf_path)
        text = ""
        for page in doc:
            text += page.get_text("text") + "\n"
        return text.strip()
    except Exception as e:
        print(f"🚩 Error processing {pdf_path}: {e}")
        return ""

# Function to analyze the resume based on job description
def analyze_resume():
    input_prompt = f"""
    Resume Text: {resume_text}

    Analyze the resume and provide a structured output in JSON format:
    1. Full Name: Extract the candidate's full name.
    2. Latest Degree: Extract the highest degree obtained.
    3. Total Experience: Extract years/months of industry experience.
    4. Candidate Summary: Write a 50-word summary of the candidate.
    5. Technical Skills: Identify key technical skills and rate them (1-5).
    6. Soft Skills: Identify key soft skills and rate them (1-5).
    7. Suitable Roles: Suggest 2 most suitable job roles.
    8. Industry Experience: Extract the company & its domain.
    9. Certifications: List any certifications found.
    10. Missing Skills: Identify skills missing based on the job description.
    11. Fit Score: Rate suitability for the job (0-100%).
    12. Recommendation: Provide overall feedback on the candidate's suitability.

    Job Description: {job_description}
    """

    model = genai.GenerativeModel('gemini-1.5-pro-latest')
    response = model.generate_content(input_prompt)
    return response.text if response and hasattr(response, 'text') else "🚩 No response received."

# Function to generate a cover letter
def generate_cover_letter():
    input_prompt = f"""
    Using the following candidate analysis and job description, generate a personalized cover letter.

    Candidate Analysis: {analyze_resume()}
    Job Description: {job_description}

    The cover letter should be professional, concise, and highlight the candidate's strengths relevant to the job.
    """

    model = genai.GenerativeModel('gemini-1.5-pro-latest')
    response = model.generate_content(input_prompt)
    return response.text if response and hasattr(response, 'text') else "🚩 No response received."

# Function to provide resume improvement suggestions
def recommend_resume_improvements():
    input_prompt = f"""
    Analyze the following resume text and provide detailed improvement suggestions.
    The suggestions should focus on better ATS (Applicant Tracking System) optimization, keyword inclusion, and formatting.

    Resume Text: {resume_text}

    Provide structured recommendations.
    """

    model = genai.GenerativeModel('gemini-1.5-pro-latest')
    response = model.generate_content(input_prompt)
    return response.text if response and hasattr(response, 'text') else "🚩 No response received."

# Function to calculate Fit Score
def calculate_fit_score():
```

```
Evaluate how well this resume fits the given job description. Assign a percentage Fit Score (0–100%) based on keyword matching, required skills, experience, and overall suitability.

Resume Text: {resume_text}
Job Description: {job_description}

Return only the Fit Score as a percentage.
"""

model = genai.GenerativeModel('gemini-1.5-pro-latest')
response = model.generate_content(input_prompt)
return response.text if response and hasattr(response, 'text') else "⚠️ No response received."

# Interactive Chatbot
def chatbot_interface():
    global resume_text, job_description # Access global variables

    print("\n--- Resume Analyzer Chatbot ---")

    # User uploads resume & job description only once
    if resume_text is None:
        print("\n📄 Upload Resume (PDF Format)")
        uploaded = files.upload()
        resume_file = list(uploaded.keys())[0]
        resume_text = extract_text_from_pdf(resume_file)

    if job_description is None:
        job_description = input("\n📄 Enter the Job Description: ")

    # Chatbot Menu
    while True:
        print("\nSelect an option:")
        print("1 Analyze Resume")
        print("2 Resume Improvement Recommendations")
        print("3 Generate Cover Letter")
        print("4 Calculate Fit Score")
        print("5 Exit")

        choice = input("\nEnter your choice (1-5): ")

        if choice == "1":
            print("\n🔍 Resume Analysis:\n", analyze_resume())

        elif choice == "2":
            print("\n💡 Resume Improvement Suggestions:\n", recommend_resume_improvements())

        elif choice == "3":
            print("\n✉️ Generated Cover Letter:\n", generate_cover_letter())

        elif choice == "4":
            print("\n📊 Fit Score:", calculate_fit_score())

        elif choice == "5":
            print("👋 Exiting Resume Analyzer. Have a great day!")
            break

        else:
            print("⚠️ Invalid choice. Please enter a number between 1 and 5.")

# Start the chatbot
chatbot_interface()
```

```
🔄 --- Resume Analyzer Chatbot ---

Select an option:
1 Analyze Resume
2 Resume Improvement Recommendations
3 Generate Cover Letter
4 Calculate Fit Score
5 Exit

Enter your choice (1-5): 1

🔍 Resume Analysis:
```json
{
 "Full Name": "Robert Conti",
 "Latest Degree": "Bachelor of Arts in Industrial Design",
 "Total Experience": "13+ years",
 "Candidate Summary": "Robert Conti is a Senior Business Analyst with over 13 years of experience at Rizonet Consulting. He specializes in aligning organizational goals with data",
 "Technical Skills": {
 "Financial Modeling": 4,
 "Business Intelligence Tools": 4,
 "Data Analysis": 4,
 "Process Enhancement": 4
 },
 "Soft Skills": {
 "Stakeholder Management": 4,
 "Communication": 3,
 "Leadership": 4,
 "Problem-Solving": 4
 },
 "Suitable Roles": [
 "Senior Business Analyst",
 "Lead Business Analyst"
],
 "Industry Experience": {
 "Company": "Rizonet Consulting, LLC",
 "Domain": "Consulting"
 },
 "Certifications": [],
 "Missing Skills": [
 "Specific software/tool proficiency (e.g., SQL, Tableau, Power BI) – The resume mentions using BI tools, but listing specific ones would strengthen it.",
 "Industry-specific knowledge – Depending on the target business domain, mentioning relevant industry expertise would be beneficial.",
 "Project Management methodologies (e.g., Agile, Scrum) – This is often sought after in Business Analyst roles."
],
 "Fit Score": "75%",
 "Recommendation": "Robert Conti possesses significant experience in business analysis and demonstrates a strong understanding of core concepts. His long tenure at one company mi
}
```

Select an option:
1 Analyze Resume
2 Resume Improvement Recommendations
3 Generate Cover Letter
4 Calculate Fit Score
5 Exit

Enter your choice (1-5): 5
```

Start coding or [generate](#) with AI.